C - Advanced Logical Reasoning - 100

① int a=5, b=3;
printf("%d", a&b);

o/p 1

② int x=6;
int result = x/3;
Printf("%d", result);

o/p 7

③ int x=4;
Printf("%d", x ^ x);

o/p 0

④ int x=1;
x = x<<2;
Printf("%d", x);

o/p 4

⑤ int x=8;
x = x>>1;
printf("%d", x);

o/p 4

⑥ ~5 equals to

o/p -6

⑦ Check if the 3rd bit in x='10' is set

o/p set

⑧ Swap two numbers using XOR

a ^ b
b ^ a
a ^ b

⇒ o/p

(9) What does x & ~(1<<n) do?

x=10 10 & ~(1<<3)
n=3

o/p = 2.

(10) How to set the nth bit of a number?

x | (1<<n)

Logical Operations

(11) int a=0; b=5;
printf("%d", a&&b);

o/p=0

(12) int a=1; b=0;
printf("%d", a||b);

o/p=1

(13) !0 equals to ?

o/p = 1

(14) int x=0;
printf("%d", x||++x);

o/p=1

(15) Difference between '&' and "&&" in C?

& - bit operation
&& - multiplication of given numbers.

(16) what is the output of 10 && 1||1?

10 && 1 || 1

o/p = 1

(17) Evaluate !(a&&b) == !a || !b

o/p = True.

(18) what is precedence && or ||?

o/p = True

(19)
```
int a=1;
int b=1;
if (a-- && ++b)
printf("%d, b);
```

o/p = 2

(20) can short-circuit evaluation prevent seg fault?

⇒ Yes, it will come:

(21) what is 1's complement of 0101?

o/p ⇒ 1010

(22) 2's complement of 0101?

o/p ⇒ 1011

(23) Find -5 in binary using 2's complement?

(24) Is ~x+1 equal to -x?

o/p = 5 | Yes

(25) Convert -10 to 2's complement in 8bit 0101

o/p = 1111 0110

(26) signed → directly to value
unsigned ⇒ Indirectly to take value only

(27) Overflow detection in 2's complement addition

⇒ 1 = binary value
⇒ 1's = complement
⇒ 2's = complement

(28) How is Subtraction done using 2's Complement? ✓

− 1 +

A − B   0101
A = 5 , B = 3
0101
1101
10010 = 2

(29) what is the result of ~(-1)?

it takes first 8 bit

O/P = 0

(30) 1's Complement of −8 in 8 bit system

O/P ⇒ 11110111

Array

(31)
```
int arr[] = {1,2,3};
printf("%d", arr[1]);
```
O/P ⇒ 2

(32) Accesing out of bounds : arr[10] = 5)

undefined behaviour

```
int arr[11]
    arr[10]=5
```

(33) diff between arr and &arr

arr = declare of the array

&arr = address of the array

(34) size of array using size of operator:

= size of (array) / size of operator (array [0]);

(35) `int a[5] = {0};`

what is a[4]?

O/P = 0

(36) How to pass array of function?

```
void fun( arr[10] , int arr[2] )
   {}
main(){
      fun()
   }
```

(37) can you modify an array in function?

Yes

(38) what does *(arr+1) mean?

value mean

(39) int a[2][2] = {{1,2},{3,4}};

printf("%d", a[1][0]);

o/p: 3

(40) Array of pointers vs pointer to array?

Array of pointer ⇒ int *arr[5];

(it holds an array 5 pointer)

Pointer to an array

int (*ptr)[5];

ptr is a single pointer that points of an entire array of 5 integers.

Use case: Jagged arrays, passing full array to functions (&arr to [])

(41) What is Linkedlist?

* It is an linear data structure made of nodes where each node
* Data
* A pointer to the next node.
* It is stored non-contiguous memory, unlike array it is dynamic in size, meaning you can grow or shrink it during runtime.

(42) How to Traverse a LinkedList:

* To traverse
* Start the head node
→ move node by node using the next pointer
* Stop when the pointer becomes Null
This helps access all data in the list one by one.

```
void traverse (struct Node *head) {
    struct Node *temp = head;
    while (temp != NULL) {
        printf ("%d", temp->data);
        temp = temp->next;
    }
}
```

---

(43) Detect Loop in LinkedList?

Use Floyd's cycle detection Algorithm.
* Take two pointers: slow and fast
* move slow one step, fast two step
* If they ever meet, a loop exist
* If fast reaches NULL, no loop is present.

```
int hasloop (struct Node *head) {
    struct Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast)
            return 1;
    }
    return 0;
}
```

---

(44) Insert Node at End in Singly Linked List

To insert at end:
* Create a new node
* If list is empty, make it head
* Else, traverse till the last node and point its next to the new node that is the node at the end.

```
void insertEnd (struct Node **head, int val) {
    struct Node *newnode = malloc (sizeof (struct node));
    newnode->data = val;
    newNode->next = NULL;
    if (*head == NULL) {
        *head = newNode;
    } return;
    struct Node *temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}
```

---

(45) Delete Node in Singly Linked List:

* Search for the node using a loop
* Keep track of the previous node
* Change prev->next to skip the deleted node
* Free the memory.
This removes a node safely.

```
void deleteNode (struct Node **head, int key) {
    struct Node *temp = *head, *prev = NULL;
    if (temp != NULL && temp->data == key) {
        *head = temp->next;
        free (temp);
        return;
    }
    while (temp && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return;
    prev->next = temp->next;
    free (temp);
}
```

(46) Reverse a Singly LinkedList:

* Start from head
* For each node, point next to previous node
* Keep track using 3 pointers: prev, curr, next
* At the end, prev becomes the new head.

```
struct Node* reverseList(struct Node* head) {
    struct Node *prev = NULL, *curr = head, *next = NULL;
    while (curr) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

(47) Difference: Singly vs Double Linked List

Feature	Singly List	Doubly List
Links	one (next)	two (prev, next)
Traverse	forward only	both directions
memory	less	more (extra pointer)
Reverse	Hard	Easy

(48) Node definition with typedef

We use typedef to simplify struct usage, instead of writing struct Node*, you can just write Node*; Improves code readability and ease.

(49) Free all Nodes in Linked List

```
void freeList(Node* head) {
    Node *temp;
    while (head) {
        temp = head;
        head = head->next;
        free(temp);
    }
}
```

(50) Why use Linked List over Arrays?

LinkedList	Array:
Dynamic size (no need to size)	Fixed size or costly to resize
Insertion/deletion is easy	costly insert/delete (shifting)
No memory waste	May allocate extra unused space
No random access	Fast random access via index

Queue

(6.1) What is FIFO?

FIFO stand for First In, First Out. It a FIFO structure like a queue. the elements that is inserted first is removed first.
Eg: standing in a queue.

(6.2) Array-based Queue Implementation

An array to store elements.
two indexes:
⇒ front → points to the elements to be removed
⇒ rear → points to the position to insert new element
When inserting (enqueue) or (removing) (dequeue); you update these indices.

```
#define size 5
int queue[size];
int front = -1, rear=
then rear++;
    front++;
```

(6.3) ~~Enqueue and dequeue functions (theory)~~ what causes queue overfl

Queue overflow happens when:
⇒ you try to insert (Queue) an element but the array is already full
⇒ Real order has reached the maximum size (rear = size-1)
this happens in static array-based queues.

(6.4) Enqueue and dequeue functions (Theory).

Enqueue:
* check for overflow
* Insert at rear index
* update rear = rear+1

Dequeue:
* check for underflow
* Remove from front index
* Update front = front+1

If front > rear the queue is empty (underflow).

```
void enqueue(int val){
  if (rear == size-1)
    printf("Queue overflow");
  else {
    if (front==-1) front=0;
    queue[++rear]= val;
  }
}

voide dequeue(){
  if (front == -1 || front > rear)
    printf("queue overflow\n");
  else
    printf("Dequeued, %d\n",
    queue[front++];
}
```

(55) Circular Queue Logic:

* In a circular queue;
  The end of the array connects to the front (like a circle)
  rear and front wrap around using modulus operator
  This avoids wasting space when some front elements are dequeued.

This avoid wasting space when some front elements are dequeued.

Index logic:

⇒ Enqueue rear = (rear+1) % size

⇒ Dequeue front = (front+1) % size.

```
rear = (rear+1) % size;
front = (front+1) % size;
```
↗ (formula of circular queue)

(56) Detect Full in Circular Queue:

(front == (rear+1) % size)) :- circular queue is full ef

This means:

⇒ Rear is just before front in circular fashion → no space to insert

⇒ Queue looks full even if technically some earlier slots are empty.

(57) Queue vs Stack:

Feature	Stack	Queue
Principle	LIFO (Last In First Out)	FIFO (First In First Out)
Insertion	At the top (push)	At the rear (Enqueue)
Deletion	From the top (pop)	From the front (Dequeue)
Usage	Function calls, undo, recursion	Scheduling, buffers, pointer queues
Memory	can be array or linkedlist	can be array or linked list.

(58) Use Queue to Reverse String?

a queue, is not ideal for reversing - because it's FIFO
we use a stack for reversing because it's LIFO

⇒ Queue can't reverse directly because first in comes out first

⇒ using two queues or combining with a stack, it can be achieved.

⇒ Use stack to reverse string - push characters, then pop to reverse.

(59) Queue using Linkedlist

⇒ Each node stores data + pointer to next node
Maintain two pointers:

front → where you dequeue
rear → where you dequeue.

Advantages:

⇒ Dynamic size (no overflow unless memory is full)
⇒ no shifting like arrays.

(60) Time Complexity of Enqueue:

Operation	Complexity	why?
Enqueue	O(1) (Constant Avg)	Just add at rear
Dequeue	O(1)	Just remove from front.

⇒ Array (Circular queue with proper tracking)
⇒ Linked List (rear for enqueue, front for dequeue).

function pointer and callback:

(61) Declare pointer to function returning int

```
int (*fptr) (int , int);
```

this declares pointer fptr to a function that takes two int's and returns int.

(62) How to pass func() pointer to another function?

You can pass it like any argument:

```
void operate (int x, int y, int (*func)(int, int));
```

Then call inside:

```
int result = func(x, y);
```

(63) callback function example in C

A callback is a function passed as an argument, usually executed later.

```
void greet (void (*cb)()) {
    cb();
}
void hello() {
    printf("Hello from callback!\n");
}
int main() {
    greet(hello);
}
```

(64) Difference: Normal vs callback function:

Normal func()	callback func()
called directly	passed as argument to another function
executes immediately	executes later indirectly
No indirection	requires function pointer.

(65) Array of function pointers:

```
int (*ops[3])(int, int);
```

Each element in ops[] can store address of a function like add, sub, mul.

(66) use function pointer to call add(int, int)

```
int add(int a, int b) {
    return a+b;
}
int (*fptr)(int, int) = add;
int result = fptr(5, 3);
```

(67) Can function pointers point to static functions?

Yes, function pointer can point to static functions,

* You can only access them within the same file
* This respects the static functions limited visibility.

(68) What is typedef for function pointer?

```
typedef int (*operation)(int, int);
operation op = add;
```

Now, you use operation as a type name of any matching function.

(69) How is function address stored in memory?

* Functions are stored in code / text segment of memory
* A function pointer holds the starting address of the function's code.
* calling the pointer like fptr() tells CPU to jump to that memory address and execute.

(10) Void (*fptr)(); what is this?

This declares fptr as a pointer to a function that;

* Take no arguments
* Return void
* Example

```
Void display() {
    printf("Hi da, macha \n");
}
int main() {
    void (*fptr)() = display;
    fptr();
}
```

Stack:

(11) What is LIFO?

LIFO stands for Last In First Out,

In this structure, the last element pushed in to the stack is the first one popped out.

It works like a stack of plates – last placed plate is removed first.

(12) Implement stack using Array

In an array-based stack, we use;

=> An array to hold elements

=> A variable top to track the index of the topmost element.

Basic operations;

push: Insert at top+1

pop : Remove from top

(13) What is Stack Overflow?

* You try to push an element, but the stack is already full.
* In arrays: when top == size - 1
* In recursion: too many function calls fill up the call stack.

(14) Recursive function and stack:

Every time a function calls itself recursively,

* A new stack frame is created in the call stack
* Each frame stores local variables, return address
* If recursion is too deep → can cause stack Overflow.

(15) push and pop Operations:

push(x):

⇒ check if stack is full.

* top++, then stack[top] = x

pop():

* Check if stack is empty
* Return stack[top], then top--

These are the basic stack Operations.

(16) Stack in Expression Evaluation:

* Infix to postfix conversion
* postfix evaluation

The stack helps hold operands or operator during the parsing and calculation process.

(17) Call stack frame in Recursion:

* pushes a new frame onto the call stack
* Contains: function parameters, local vars, return address
* when function returns, its frame is popped off.

This is why stack is critical in recursion.

(18) Stack using Linked List:

* Each node contains data + pointer to next
* Stack operations are done at the head (for O(1) time)
* no size limit → avoids fixed-size issue in array based stack.

(19) check stack overflow:

* you can try to pop an element from an empty stack

check if (top == -1) → underflow condition

underflow means there's nothing to remove.

(20) Time complexity of push/pop?

Operation	Time Complexity
push	O(1)
pop	O(1)
peek	O(1)

Both array and LinkedList stack implementations offer constant for basic operations.

(81) malloc () vs calloc ()

Feature	malloc ()	calloc ()
Full form	Memory allocation	Contiguous allocation
Syntax	malloc (n * size of (type))	calloc (n * size of (type))
Initialization	does not initialize memory	Initialize to zero
Speed	Slightly faster	slightly slower (due to zeroing)
Use case	when manual initialization preferred	when zeroed memory is needed.

(82) What is segmentation fault?

* A runtime error when a program access memory it shouldn't

* Dereferencing a NULL pointer
* Accessing array out of bounds
* Writing to read-only memory
* using uninitialized or freed pointer.

(83) Accessing freed memory?

* Happens when pointer memory is freed, but you still use it.
* called dangling pointer.

```
int *p = malloc(4);
free(p);
*p = 10;
```

may crash, corrupt data, or silently fail.

(84) Double free error:

calling free() on same memory twice

```
free(p);
free(p);
```

It will crash or heap corruption, tool like valgrind to detect.

(85) Memory Leak Example

Allocated memory is never freed, causing program to consume more memory.

```
void leak() {
    int *p = malloc(100);
    // no free(p) → leak happens.
```

(86) Dangling pointer Usage:

pointer pointing to freed or out-of-scope memory.

```
int *func(){
    int x = 10;
    return &x; // x dies memory gone after return
```

3.

using it = undefined behavior, Always nullify after free.

(87) Heap vs Stack Memory:

Feature:	Stack	Heap
Allocation	Automatic (fun call)	Manual (malloc/calloc)
deallocation	Automatic	Manual (free)
size	Small, fixed	Large, dynamic
Speed	Faster	Slower
Errors	Stack overflow	Memory leak, corruption.

(88) Segfault on NULL pointer dereference:

```
int *p = NULL;
*p = 5;
```

⇒ Accessing memory at address 0x0 is illegal,

⇒ OS protects this page → segfault

Always check pointer != NULL before use.

(89) Use-after-free vulnerability:

⇒ Accessing memory after it was freed.

⇒ Security issue, attacker can inject fake data or exploit.

```
char *data = malloc(100);
free(data);
strcpy(data, "Exploit"); // use-after-free
```

(9) Tools to Detect Memory Errors:

Tool	Use
valgrind	detects memory leaks, use-after-free
ASAN	Address sanitizer (fast runtime checker)
gdb	debugger to trace segfaults
electric fence	detects buffer overflows
clang sanitizers	Modern toolchain for runtime checks.